

用户中心集成说明文档

版本号	修订人	创建日期	变更概要
1.0	汤龙波	2019/12/05	签名规则升级
1.1	蒋晓满	2020/05/08	添加用户中心相关接口
1.1.1	翟献成、蒋晓满	2020/05/13	完善文档说明 增加 Kafka 消息说明

目录

1	概述	4
1.1	编写目的	4
1.2	交互模式	4
1.3	接口地址	4
1.4	用户中心分配的全局参数	4
1.4.1	前端交互相关参数	4
1.4.2	后台交互相关参数	5
2	集成流程说明	5
2.1	用户登录	5
2.1.1	用户登录流程示意图	5
2.1.2	用户登录过程时序图	7
2.2	服务端认证	8
2.2.1	交互流程	8
2.2.2	获取 ticket	9
2.2.2.1	获取 ticket 流程	9
2.2.2.2	ticket 内容解密	11
2.2.2.3	ticket 解密示例 demo	12
2.2.3	刷新 ticket	12
2.2.4	业务接口调用	13
2.2.4.1	请求体之公共参数签名流程	13
2.2.4.2	请求体之业务参数 SM4 加密算法流程	14
2.2.4.3	业务请求示例	15
2.2.5	通用参数	17
2.2.6	通用 util 工具	18
2.2.7	总结	18
3	接口定义说明	19
3.1	服务端认证接口	19
3.1.1	获取 ticket 接口文档	19
3.1.2	刷新 ticket 接口文档	20
3.2	业务接口定义	20
3.2.1	根据校验码获取用户信息	20

3.2.2	通过帐密获取用户信息	22
3.2.3	校验会话的有效性	23
3.2.4	退出登录	24
3.2.5	根据登录用户获取员工信息	25
3.2.6	获取指定租户对应信息	27
3.2.7	获取区域平台所有机构	28
3.2.8	获取用户信息	31
3.2.9	获取员工信息	32
3.2.10	获取部门信息	35
3.2.11	切换登录租户	37
3.3	消息中心(Kafka)主题和消息体	38
3.3.1	用户消息推送	38
3.3.2	租户消息推送	39
3.3.3	员工消息推送	40
3.3.4	部门消息推送	40
4	全流程调用过程图解	41
4.1	获取 ticket 示例	41
4.1.1	地址及 body	41
4.1.2	注意 header 不能错	42
4.1.3	对结果的处理	42
4.2	刷新 ticket 示例	42
4.3	业务接口调用示例	43
5	字典说明	44
6	业务响应码说明	46
7	其他	46

1 概述

1.1 编写目的

此文档主要用于用户中心集成方案说明，帮助开发者快速实现对接，包含接口数据传输加密说明、接口请求方式说明及接口对接出入参说明。

1.2 交互模式

集成接口采用 HTTPS 协议封装数据，以 Restful 数据格式交互，开发者需以 POST、GET 方式发起接口请求，POST 参数采用 json 格式封装。

1.3 接口地址

测试环境（后台）：待定

测试环境（统一登录页面）：待定

生产环境（后台）：待定

生产环境（统一登录页面）：待定

1.4 用户中心分配的全局参数

1.4.1 前端交互相关参数

- 1、clientId：业务系统标识 ID， 全局唯一
- 2、realm：业务系统所属域， 如 管理端(A)、机构端(B)、居民端(C)
- 3、terminal：业务系统终端类型， 如 Web、H5、APP 等

使用方式见后续 接口定义 说明。

1.4.2 后台交互相关参数

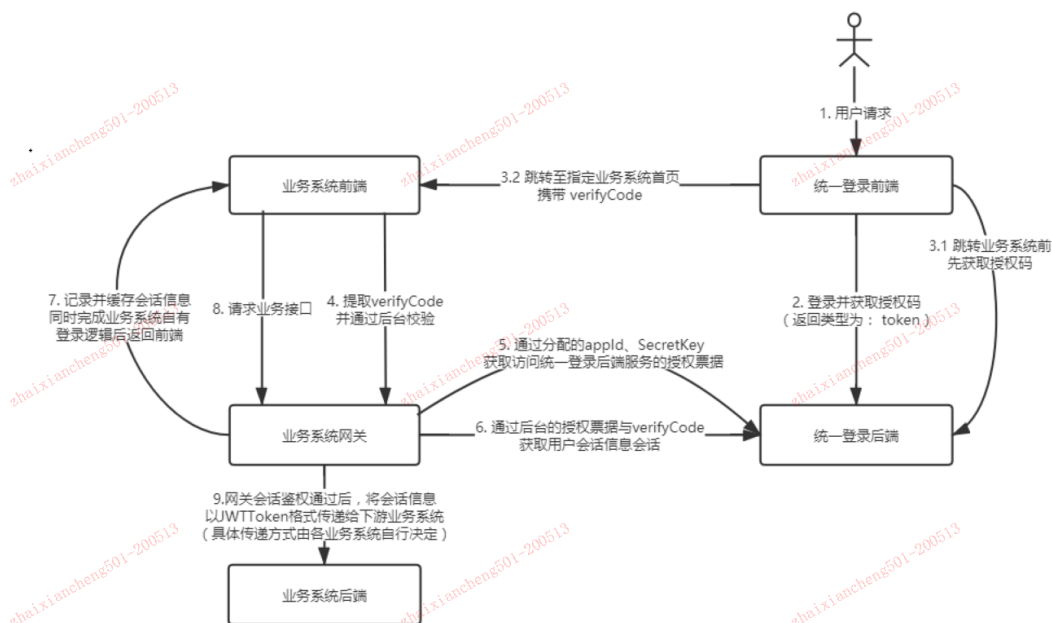
- 1、appld：授权应用唯一 ID，不可更改
- 2、appSecret：授权应用 appld 对应的密钥内容串，密钥泄露会带来严重的系统安全风险问题，需要客户端妥善保管，

2 集成流程说明

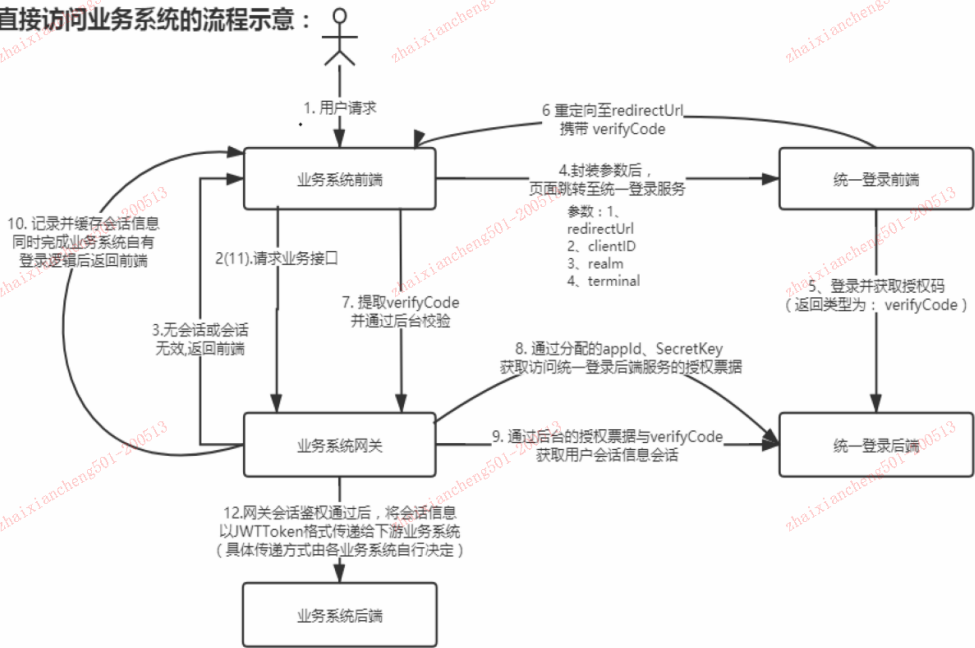
2.1 用户登录

2.1.1 用户登录流程示意图

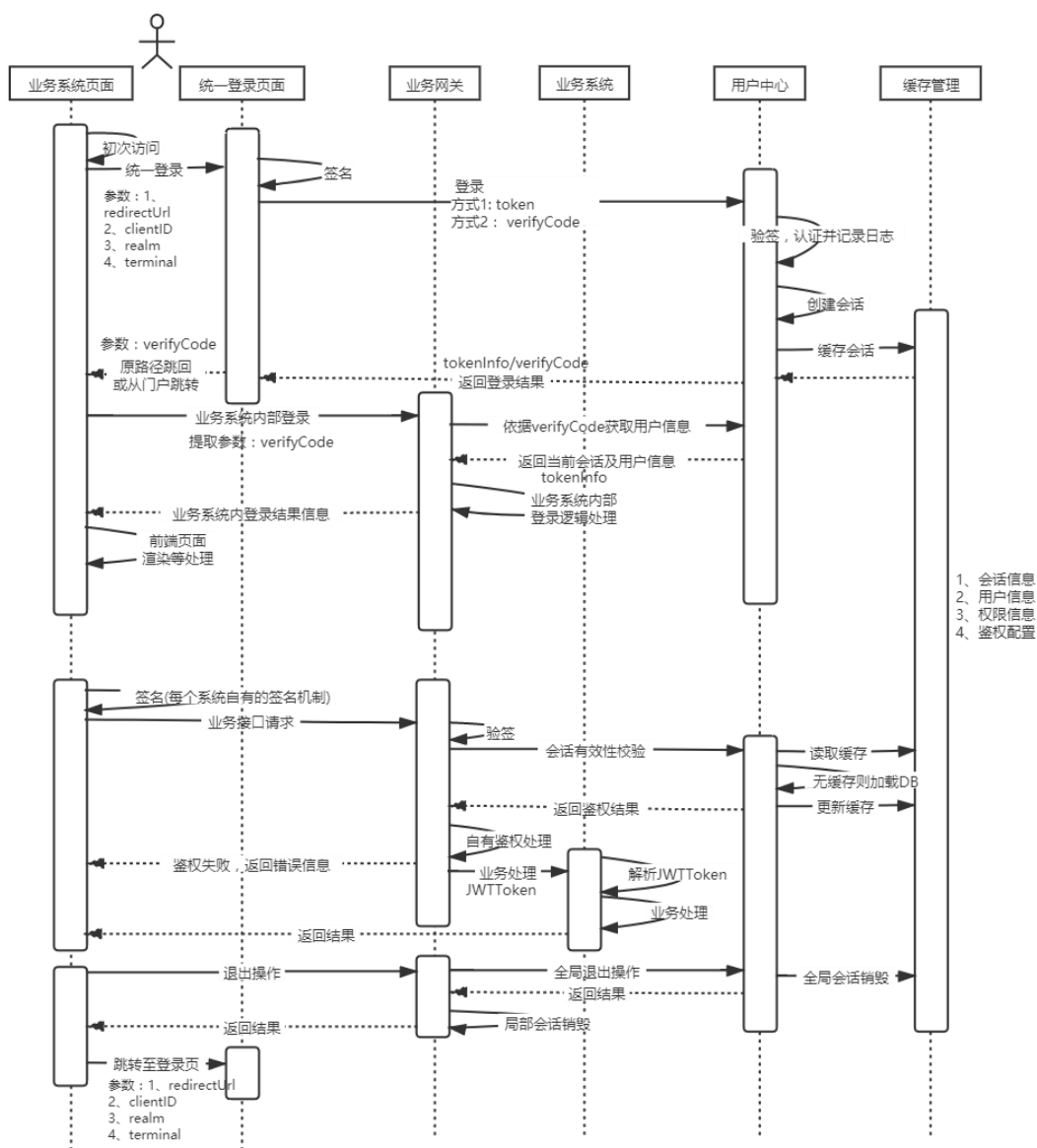
用户通过统一门户访问业务系统的流程示意：



用户直接访问业务系统的流程示意：



2.1.2 用户登录过程时序图



说明：

1、用户访问系统有两种情况

- 1) 访问统一登录页面，登录成功后携带授权码从统一门户跳转至业务系统。
- 2) 直接访问业务系统功能页面，业务系统拦截后携带必要的参数导航到统一登录页面，登录成功后自动回调至原业务系统功能页面内。

2、相关参数定义说明

- 1) redirectUrl: 登录成功后跳转地址
- 2) clientId: 业务系统标识 ID，由用户中心分配
- 3) realm: 业务系统所属域，由用户中心分配
- 4) terminal: 业务系统终端类型，由用户中心分配

5) verifyCode: 用户中心颁发的临时校验授权码，通过该授权码可以获取用户和会话信息

6) tokenInfo: 会话数据体，核心属性如下（详见接口文档）

accessToken: 短期会话 Token，有效时长短，页面与后端交互时用于识别会话的依据

refreshToken: 全局会话 Token，有效时长久，当 AccessToken 过期时用于刷新 accessToken

accessExpireDate: accessToken 有效期

refreshExpireDate: refreshToken 有效期

apiKey: 动态签名盐值，

userInfo: 用户相关信息，包括用户 ID、用户名、租户 ID、租户名称等

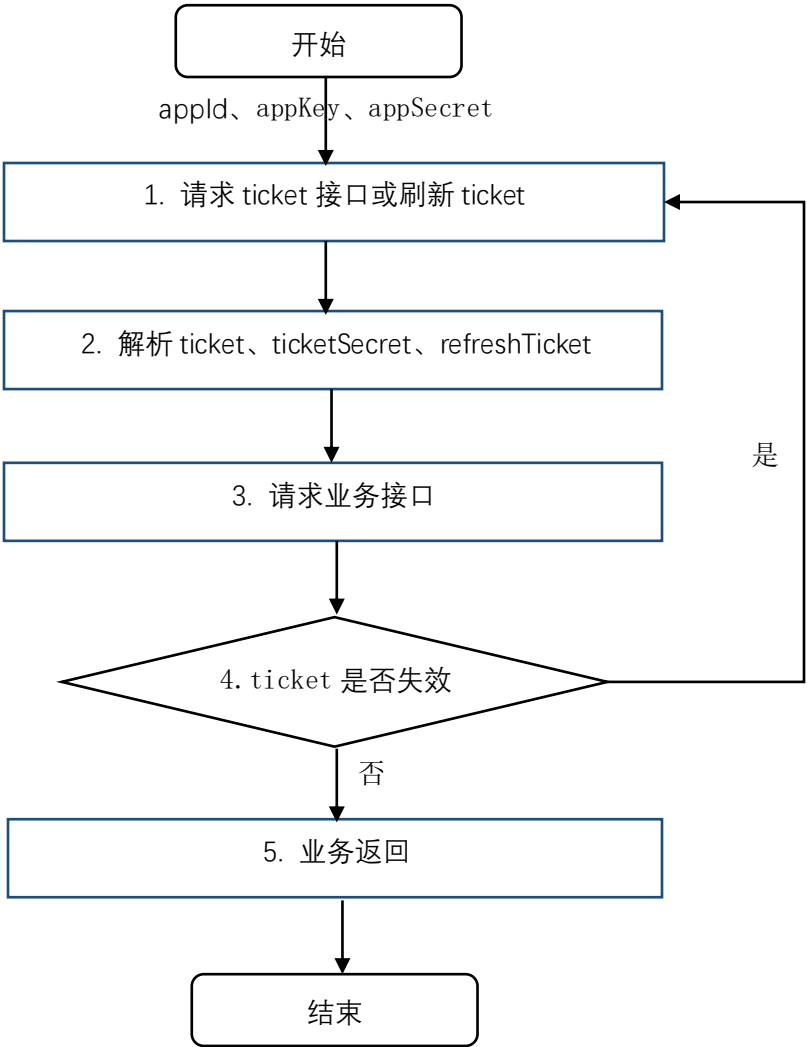
3、后台接口认证过程参考 2.2 章节内容

2.2 服务端认证

2.2.1 交互流程

接口调用流程如下，其中 appId、appKey、appSecret 由用户中心提供，ticket、

ticketSecret 需通过请求接口获取。



2.2.2 获取 ticket

2.2.2.1 获取 ticket 流程

应用方获取 ticket 整体流程如下图 1 所示，整体步骤分为三步：

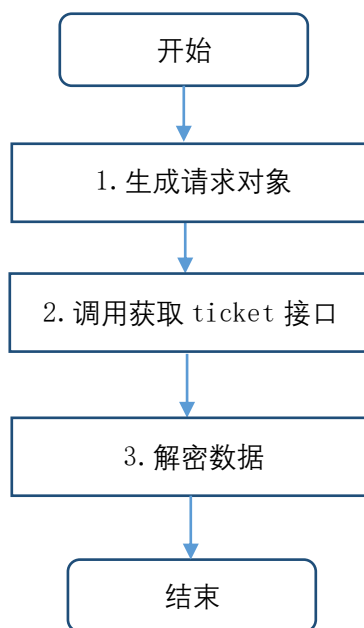


图 1. 获取 ticket 流程图

第 1 步：组装请求对象

第 2 步：接口请求

第 3 步：使用 appSecret 进行解密

2.2.2.2 ticket 内容解密

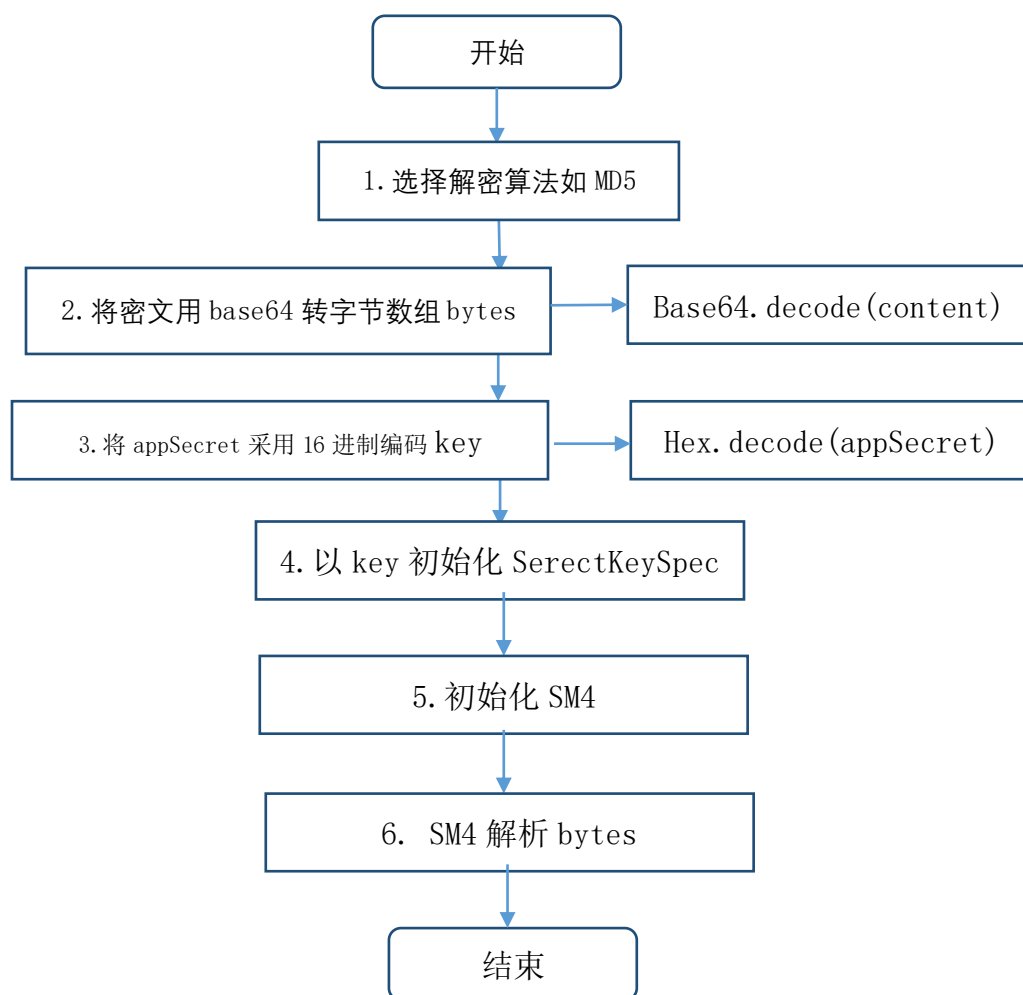


图 3. ticket 解密流程图

解密调用 SignSignUtil 的 decryptData 方法实现解密，采用国密 SM4 算法

方法名：

decryptData ()

参数：

String appSecret //对称加密密钥

String cipherText //加密数据

2.2.2.3 ticket 解密示例 demo

1、返回结构示例：

```
{
  "data": "7TPXU9hAYxmebg1xcqoPqexK3ViaSLQTA/CotHTziv7/Ho8nfNCt38MII4Y407lpDvAdtJPITwXsVQitNnsUIUPLc/1L543ip/RyTslW6byuhd+oEE7sve041rMZSP5k",
  "retCode": "SUCCESS",
  "retMsg": "成功",
  "success": true
}
```

2、解密示例

```
private static void decodeTiketRes() {
    String appSecret = "####";
    String x =
        "7TPXU9hAYxmebg1xcqoPqexK3ViaSLQTA/CotHTziv7/Ho8nfNCt38MII4Y407lpDvAdtJPITwXsVQitNnsUIUPLc/1L543ip/RyTslW6byuhd+oEE7sve041rMZSP5k";
    String decodedTicketDTOStr =
        SignUtil.decryptData(appSecret, x);
    AuthAccessRespDTO ticketResponseDTO =
        JSONObject.parseObject(decodedTicketDTOStr,
            AuthAccessRespDTO.class);
    log.info("获取ticket请求参数ticketResponseDTO
        {} ", JSONObject.toJSONString(ticketResponseDTO));
}
```

2.2.3 刷新 ticket

每个 ticket 默认有效时间是 15 分钟，应用方可通过传入 refreshTicket 来重置 ticket 的有效时间。该请求过程不需加密，直接传入 refreshTicket 进行 POST 请求即可。详情参见接口文档。

2.2.4 业务接口调用

每一个业务请求体包含两个部分，公共参数和业务参数。

- 1) 生成公共请求参数签名可以采用 MD5 或者 SM3 算法，签名因子需要根据获取的 ticket/ticketSecret 作动态签名，放在 header。
- 2) 业务参数的封装要使用 SM4 算法。

2.2.4.1 请求体之公共参数签名流程

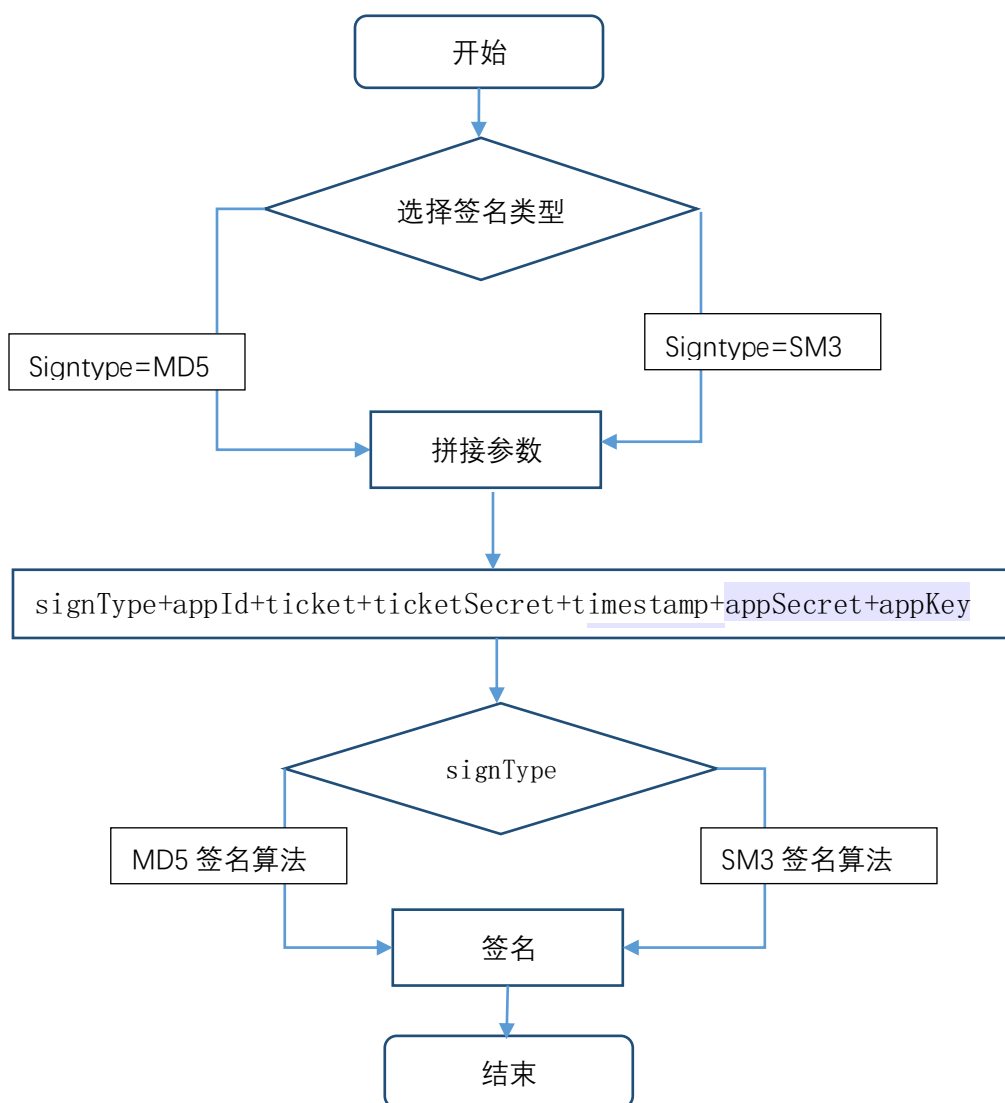


图 3. 业务请求签名流程图

业务请求参数封装示例

Header 参数:

```
"appId": "##",
"sign": "##",
"signType": "MD5",
"ticket": "KMsETnNQHvc jInWzIxEKSoGaniLXKbFf",
"timestamp": "1574999522746",
"requestId": "123456789"
```

POST 请求 body 参数

```
{
  "encodeData": "d2pUyKFU5Dgs/rYWplaZPppoGZMZ2kJ6n9+pVQISA/I="
}
```

GET 请求携带参数

域名?encodeData= d2pUyKFU5Dgs/rYWplaZPppoGZMZ2kJ6n9+pVQISA

2.2.4.2 请求体之业务参数 SM4 加密算法流程

业务请求参数加密采用的是国密 SM4 算法，SM4 算法流程图如下图 4 所示：

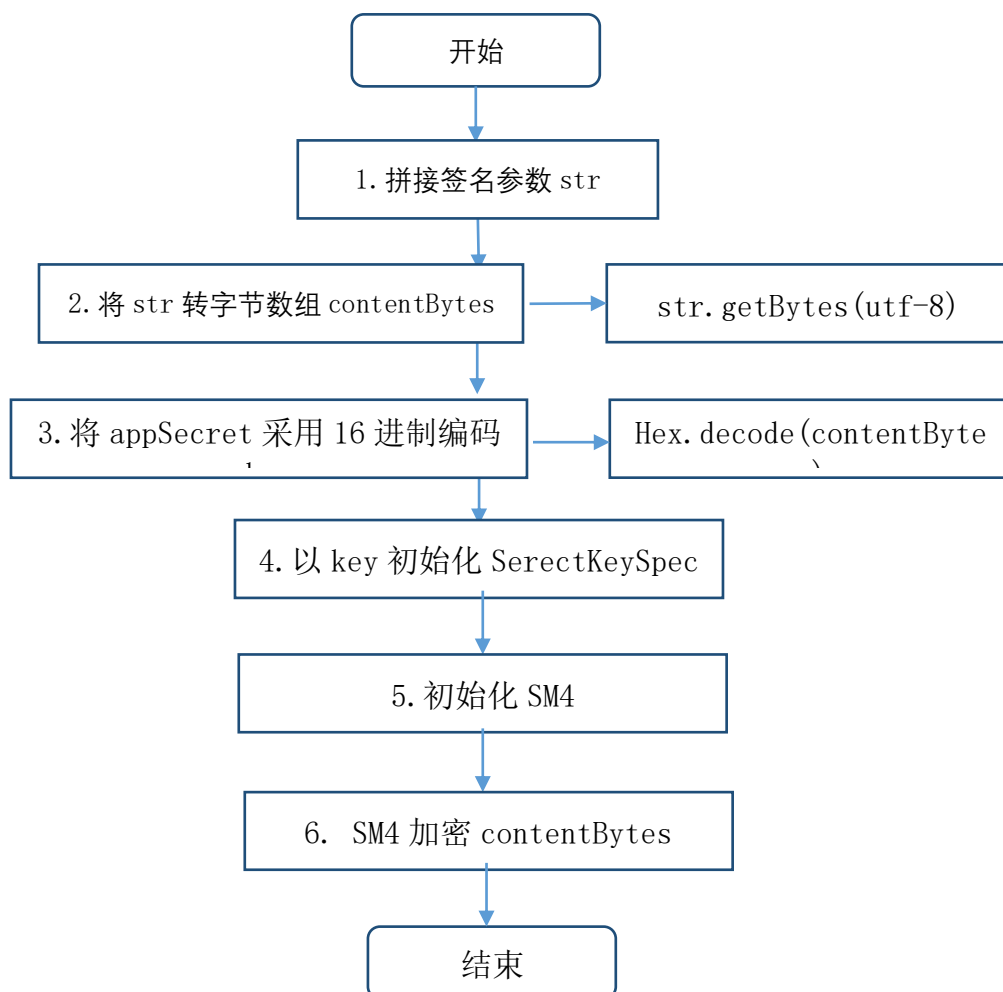


图 4. SM4 签名算法流程图

备注：SM4 算法模式采用分块模式 SM4/ECB/PKCS5Padding

2.2.4.3 业务请求示例

1、POST 请求示例

```
@PostMapping("/testPost")
public String testPost() throws Exception {
    RestTemplate restTemplate = new RestTemplate();
    restTemplate.getMessageConverters().add(new
StringHttpMessageConverter(Charset.forName("utf-8")));
    String appId = "##";
    String appKey = "##";
    String appSecret = "##";
    String signType = "MD5";
    String ticket = "YzEQrKbcZLGclzWlrVarfoTfrfsayTKx";
    String ticketSecret =
"oYhJdWFFaDPTcKJMZXJOZSmiRIapKPLb";
    String timeStamp = System.currentTimeMillis() + "";
    String sign = SignUtil.sign(signType,
appId,ticket,ticketSecret,timeStamp,appSecret,appKey);
    HttpHeaders heard = new HttpHeaders();
    heard.setContentType(MediaType.APPLICATION_JSON_UTF8);
    heard.add("appId", appId);
    heard.add("timestamp", timeStamp);
    heard.add("signType", signType);
    heard.add("sign", sign);
    heard.add("ticket", ticket);
    heard.add("requestId", UUID);
    JSONObject obj = new JSONObject();
    obj.put("test", 123);
    String decryptData = SignUtil.encryptData(appSecret,
obj.toJSONString());
    obj.put("encodeData", decryptData);
    HttpEntity<Map<String,Object>> entity = new
HttpEntity<>(obj,heard);
    String ret =
restTemplate.postForObject("http://127.0.0.1:8086/test/api",
entity, String.class);
    return ret;
}
```

2、GET 请求示例

```
@GetMapping("/testGet")
public String testGet() throws Exception {
    RestTemplate restTemplate = new RestTemplate();
    restTemplate.getMessageConverters().add(new
StringHttpMessageConverter(Charset.forName("utf-8")));

    String appId = "##";
    String appKey = "###";
    String appSecret = "###";
    String signType = "MD5";
    String ticket = "YzEQrKbcZLGclzWlrVarfoTfrfsayTKx";
    String ticketSecret =
"oYhJdWFFaDPTcKJMZXJOZSmiRIapKPLb";
    String timeStamp = System.currentTimeMillis() + "";
    String sign = SignUtil.sign(signType,
appId,ticket,ticketSecret,timeStamp,appSecret,appSecret,
appKey);

    HttpHeaders heard = new HttpHeaders();
    heard.setContentType(MediaType.APPLICATION_JSON_UTF8);
    heard.add("appId", appId);
    heard.add("timestamp", timeStamp);
    heard.add("signType", signType);
    heard.add("sign", sign);
    heard.add("ticket", ticket);
    heard.add("requestId", UUID);
    String ret =
restTemplate.getForObject("http://127.0.0.1:8086/test/testGet?
encodeData=123456789", String.class);
    return ret;
}
```

3、业务返回解密示例

```
private static void decryptBizRes() {
    String res
="Plkx7/10S/GiKXqUVciyCi0NugyZgnh9YgIRKXy2R5tVdsts271Fb4IMQaUf
XxJzmPhxdGtmpbuNfcrUvCSAr7TfJ8lVSDr0a0DozN2OXNEUtc+jwRBSGwB5LJ
+W/7bkLdVJsMsNd7Lo2S9ayIBqmIzYv73jq0IzaHtL4RlXI6HL9N3yTx0Jd6GY
3YWHk68FJfR7Zew7oi7Rf+1b9jQnB8s47qJTVsqFp1L1ZiBCCp0I25SsaX5sK8
xP0rEWas0gDmkvEQgMxGb52zh+PaV9aZ0MyyKnr18IArcCuAZsRkf2mxm6BX42
QmnYrLwz7T9fPabKwo065i4vihZqGy1ISXjppWE3tsmtVfWALI7WZ/wJn6z0Xy
5TTAw8d0vDSroU+39pVni37/vCuYoJnQlLn1NopY3lHmj0Jd0q93FAWrZEbrUS
```



```
s04baotjS+Eck7oqpRvhK+z39qQ0zqbPzMc3D8zUi6f5AmA3KW6ilE4Zf5fkxh
yD5zgiK5LvkmN5VRtVFLXPo8EQUhsAeSyflv+25C3VSbDLDXey6NkvWsiAapiP
AE6/09ubJoJ2TeDB6Pqiy/Td8k8TiXehmN2Fh50vBQp0ViFrB1xYeB15928uq
VGhQj1kEnM1XACJCvEfVQyCNuUrGl+bCvMTzqxFmrDoDpjP+Y07sFqVINCJTBA
A5Vg6leh/OYqqEksW7RXH3ug3MVFUr2fsVyHnvWRuB8I0smI/iw+M0LauGJ0We
yTdHV46T1hN7bJrVX1gCy01mf8P8PwTr0zH4Ab40NEYk5Df6AeoHQK/5B2LtTS
vYnejCyjZGvzsZnsIDL4g98cIOvsRq8NtQYJCuB0ywMAkRrDL/w4t845Dq9R2
c3d40xDqyFgKbg9CMHLSqtXSE0jT0/65Rs0tkTm7xZbwYtImAJCJtiLZJveEAK
S/nDjhfu5Vr9AjOVfN7dwpCK7MutGfvMuLEGlS6nV90oU7idKBSffdgwhbIaF
noT3Cjqij9Ce18J1lRBukFi+A78nHq151Yszkcx7s7pGS5jzp5VB0KcSFDICa1
fRu1jzyw/10hT8nHrxiHzIBtDv+cGtxIiK41hIH+g5ZZu5tF4+qoFMIjR3Vrbw
8xNREcaVbCsr7j/J82jHS1JDmQkfAkPFmWj4PHso2Rr87GZ7CAy+IPfHCDr3kp
/v7nQE0x2zChZJRd63uLQh60CcEilpgBf9s4Cge0e0k9YTe2ya1V9YAsjtZn/E
6IgVzLaI2t2wgMJ4BJ93p1IAQz0lh8Mrm7xnr7tEXR+738kY9A3a4BGHzPQt3i
mA=="
```

```
String appSecret = "###";
String decryptData = SignUtil.decryptData(appSecret,
res);
}
```

2.2.5 通用参数

通用参数为接口公共参数，在进行接口调用过程中，请求和响应都需带上通用参数。

HEAD 参数：

节点名称	位置	数据类型 型	必填	说明
requestId	head	String	是	请求序列号---链路追踪
timeStamp	head	String	是	时间戳—加密因子
appld	head	String	是	appld—加密因子
sign	head	String	是	签名
signType	head	String	是	签名类型—加密因子
ticket	head	String	是	票据—加密因子

网关统一响应参数：

节点名称	父节点名称	数据类型	必填	说明
success		Boolean	是	成功 true 失败 false
retCode		String	是	响应码
retMsg		String	是	响应描述
data		String	是	加密后的数据

响应码类型：

retCode：999999	retMsg：成功
retCode：GW0000	retMsg：服务器开小差，请稍后重试
retCode：GW0001	retMsg：应用未注册
retCode：GW0002	retMsg：参数错误
retCode：GW0003	retMsg：ticket 已失效
retCode：GW0004	retMsg：签名校验不通过
retCode：GW0005	retMsg：该接口无访问权限
retCode：GW0006	retMsg：接口 404 NOT FOUND
retCode：GW0007	retMsg：refreshTicket 已失效

2.2.6 通用 util 工具

 Sm4Util.java

 Md5Util.java

 SignUtil.java

 Sm3.java

 Sm3Digest.java

 ByteConvertUtil.java

2.2.7 总结

每个业务请求需要根据颁发的密钥信息来获取票据，再然后根据票据信息对请求体进行签名加密。票据信息的有效时间默认是 15 分钟，应用方需在 15 分钟内刷新 ticket 或者重

新请求 ticket，重置票据的有效使用时间。

应用方如果不是 JAVA 编码，需要自行到中央仓库下载国密 SM4 算法相关依赖。

3 接口定义说明

3.1 服务端认证接口

3.1.1 获取 ticket 接口文档

接口路径	/auth/ticket				
说明	根据 appld、appKey 获取 ticket				
请求方式	POST				
Content-Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
请求参数	appld		String	是	颁发的 APPID
	appSecret		String	是	颁发的 appSecret
	节点名称	父节点名称	数据类型	必填	说明
解密后响应参数	ticket	data	String	是	票据
	ticketSecret	data	String	是	票据密钥
	refreshTicket	data	String	是	刷新密钥

示例：

1.请求示例：

```
{
  "appld": "###",
  "appSecret": "###"
}
```

2.返回示例：

```
{
  "retCode": "999999",
  "retMsg": "成功",
  "success": true,
  "data":
  "pPqyqOIdb+QBOjflSH/cdLdBYYP/cSgtIOU9/opaibBVzO1DugVbiln3QHspGUmuE/KuR69/
  zWUANC90DCIUxCn43qPRKjC7uSqYFoe1SSCRj67KH7N7HBgO5E1yJLo"
}
```

3.解密后示例：

```
{
  "ticket": "PqECj6T7RLe6cjS5zq/Sew",
  "ticketSecret": "40wdZjl7TVIKZXWk0VmMSShPkvznfFUBAzKrs/uBFQLZ",
  "refreshTicket": "AsSsBAzKrs/uBFQLZ"
}
```

3.1.2 刷新 ticket 接口文档

接口路径	/auth/refresh				
说明	刷新 ticket				
请求方式	POST				
Content-Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
业务请求参数	refreshTicket		String	是	票据
	节点名称	父节点名称	数据类型	必填	说明
返回参数	retCode		String	是	返回编码
	retMsg		String	是	返回消息
	success		Boolean	是	是否成功
	data		Object	是	返回数据

实例：

请求示例：

```
{
  "refreshTicket": ""
}
```

返回示例：

```
{
  "retCode": "999999",
  "retMsg": "成功",
  "success": true,
  "data": ""
}
```

3.2 业务接口定义

3.2.1 根据校验码获取用户信息

接口代码	/sso/code/userInfo
说明	根据票据获取用户信息
请求方式	POST

Content - Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求参数	verifyCode		String	是	校验码
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appld	head	String	是	appld—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	object		响应对象
	accessToken	data	String		登录 token
	refreshToken	data	String		刷新 Token
	accessExpireDate	data	Integer		accessToken 有效时间
	refreshExpireDate	data	Integer		refreshtoken 有效时间
	userInfo	data	object		用户信息
	userId	userInfo	String		用户 id
	accountName	userInfo	String		账号
	realName	userInfo	String		真实姓名
	gender	userInfo	String		性别（见字典）
	nickName	userInfo	String		昵称
	tenantId	userInfo	String		租户 id
	tenantName	userInfo	String		租户 Name
	departmentCode	userInfo	String		院区编码
	departmentName	userInfo	String		院区名称

示例：

1、请求地址：<https://xxtest.city.pingan.com/openapi/ssso/code/userInfo>

2、请求报文：

```
{
  "verifyCode": "ghaaafafdfasfnfs999900099"
}
```

3、返回报文：

```
{
  "retCode": 999999,
  "success": true,
  "retMsg": "成功",
  "data": {
    "accessToken": "aaafafdfasfnfs999fff",
    "refreshToken": "djfasfnfs999fff",
    "accessExpireDate": 6000,
  }
}
```

```
"refreshExpireDate": 84600,
"userInfo": {
  "userId": "aaafafdjfasnfns999",
  "accountName": "test",
  "realName": "张三",
  "gender": 1,
  "nickName": "张思",
  "tenantId": "aaafafdjfasnfns999ff",
  "tenantName": "珠海人民医院"
}
```

3.2.2 通过帐密获取用户信息

备注：登录密码需要对明文进行 MD5 加密

接口代码	/sso/login/account				
说明	通过帐密获取用户信息				
请求方式	POST				
Content-Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求参数	username		String	是	用户名
	password		String	是	密码（MD5）
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appld	head	String	是	appld—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	clientId	head	String	是	子系统编码
	realm	head	String	是	域
	terminal	head	String	是	终端
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	object		响应对象
	accessToken	data	String		登录 token
	refreshToken	data	String		刷新 Token
	accessExpireDate	data	Integer		accessToken 有效时间
	refreshExpireDate	data	Integer		refreshtoken 有效时间
	userInfo	data	object		用户信息
	userId	userInfo	String		用户 id

	accountName	userInfo	String		账号
	realName	userInfo	String		真实姓名
	gender	userInfo	String		性别
	nickName	userInfo	String		昵称
	tenantId	userInfo	String		租户 id
	tenantName	userInfo	String		租户 Name

示例：

1、请求地址：<https://xxtest.city.pingan.com/openapi/sso/login/account>

2、请求报文：

```
{
  "userName": "test",
  "password": "80198ee19eb7ca656996dc87e821bd00"
}
```

3、返回报文：

```
{
  "retCode": 999999,
  "success": true,
  "retMsg": "成功",
  "data": {
    "accessToken": "aaafafdfasnfns999fff",
    "refreshToken": "djfasnfns999fff",
    "accessExpireDate": 6000,
    "refreshExpireDate": 84600,
    "userInfo": {
      "userId": "aaafafdfasnfns999",
      "accountName": "test",
      "realName": "张三",
      "gender": 1,
      "nickName": "张思",
      "tenantId": "aaafafdfasnfns999ff",
      "tenantName": "珠海人民医院"
    }
  }
}
```

3.2.3 校验会话的有效性

备注：若 refreshToken 没有过期而 accessToken 过期会帮忙刷新 token

接口代码	/sso/check/token
说明	校验会话的有效性
请求方式	POST

Content - Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求参数	refreshToken		String	是	刷新 token
	accessToken		String	否	登录 token
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appld	head	String	是	appld—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	object		响应对象

示例：

1、请求地址：<https://xxtest.city.pingan.com/openapi/sso/check/token>

2、请求报文：

```
{
  "refreshToken": "ca656996dc87e821bd00",
  "accessToken": "80198ee19eb7ca656996dc87e821bd00"
}
```

3、返回报文：

```
{
  "retCode": 999999,
  "success": true,
  "retMsg": "成功",
  "data": {
  }
}
```

3.2.4 退出登录

接口代码	/sso/user/logout				
说明	退出登录				
请求方式	POST				
Content - Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求	refreshToken		String	是	刷新 token

参数	accessToken		String	否	登录 token
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appld	head	String	是	appld—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	object		响应对象

示例：

1、请求地址：<https://xxctest.city.pingan.com/openapi/sso/user/logout>

2、请求报文：

```
{
  "refreshToken":"ca656996dc87e821bd00",
  "accessToken":"80198ee19eb7ca656996dc87e821bd00"
}
```

3、返回报文：

```
{
  "retCode":999999,
  "success":true,
  "retMsg":"成功",
  "data":{
  }}
```

3.2.5 根据登录用户获取员工信息

备注：根据登录用户获取对应的员工信息

接口代码	/sso/user/emplInfo				
说明	根据登录用户获取员工信息				
请求方式	POST				
Content-Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求参数	userId		String	是	用户 id
	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪

Head 参数	timeStamp	head	String	是	时间戳—加密因子
	appld	head	String	是	appld—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	Object		响应对象
	employeeId	data	String		员工 ID
	employeeName	data	String		姓名
	employeeCode	data	String		工号
	postName	data	String		性别（见字典）
	postCode	data	String		职位编码（见字典）
	titleCode	data	String		职称编码（见字典）
	titleName	data	String		职称名称（见字典）
	departmentId	data	String		所属部门 id
	departmentName	data	String		所属部门名称
	departmentCode	data	String		所属部门编码

示例：

1、请求地址：<https://xxtest.city.pingan.com/openapi/sso/user/emplInfo>

2、请求报文：

```
{
  "userId": "djfasnfns999fff"
}
```

3、返回报文：

```
{
  "retCode": "999999",
  "success": true,
  "retMsg": "成功",
  "data": {
    "employeeId": "dfdfdfd",
    "employeeName": "ceshi",
    "employeeCode": "sddff",
    "postCode": "230",
    "postName": "医生",
    "titleCode": "231",
    "titleName": "主任医师",
    "departmentId": "fgsfsg",
    "departmentName": "fgsfsg",
    "departmentCode": "fgsfsg"
  }
}
```

3.2.6 获取指定租户对应信息

备注：查询指定租户 id 的对应的租户信息

接口代码	/console/tenant/detail				
说明	查询指定租户 id 的对应的租户信息				
请求方式	POST				
Content-Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求参数	tenantId		String	是	租户 id
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appld	head	String	是	appld—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	Object[]		响应对象
	tenantId	data	String		租户 id
	tenantName	data	String		租户 Name
	tenantCode	data	String		租户编码
	realCode	data	String		领域类型
	organizationType	data	String		租户类型(机构类型 1-医疗机构 2-区域平台)
	gradeName	data	object		医院等级
	adminUser	data	String		管理员账号
	adminName	data	String		姓名
	devisionType	data	String		行政区划级别
	provinceCode	data	String		省
	cityCode	data	String		市
	districtCode	data	String		区
	tenantAddress	data	String		地址
	longitude	data	String		经度
	latitude	data	String		维度
	addDate	data	Date		创建时间 (yyyy-MM-dd HH:mm:ss)
	modifyDate	data	Date		修改时间 (yyyy-MM-dd HH:mm:ss)
	disabled	data	Integer		是否禁用 1-是 0-否
	delFlag	data	Integer		逻辑删除标志 1-是 0-否

示例：

1、请求地址：<https://xxctest.city.pingan.com/openapi/console/tenant/detail>

2、请求报文：

```
{
  "tenantId": "djfasnfns999fff",
}
```

3、返回报文：

```
{
  "retCode": 999999,
  "success": true,
  "retMsg": "成功",
  "data": {
    "tenantId": "fgffadfafd",
    "tenantName": "第一人民医院",
    "tenantCode": "hhhh",
    "realCode": "B",
    "organizationType": 1,
    "gradeName": "三级甲等",
    "adminUser": "test",
    "adminName": "降 jjll",
    "devisionType": "1",
    "provinceCode": "659001000000",
    "cityCode": "659001000000",
    "districtCode": "659001000000",
    "tenantAddress": "xxxxx 地址",
    "longitude": "121.9090",
    "latitude": "90.9999",
    "disabled": 1,
    "delFlag": 1
  }
}
```

3.2.7 获取区域平台所有机构

备注：查询区域平台下面的所有的机构列表

接口代码	/console/tenant/infos				
说明	获取区域平台所有机构				
请求方式	POST				
Content - Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求	pageSize		Integer		页内数量（默认 10）

参数	pageNum		Integer		页码（默认 1）
	startTime		Date	是	最后修改时间（yyyy-MM-dd HH:mm:ss）
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号-- 链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appld	head	String	是	appld—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	Object		响应对象
	total	data	Integer		总数
	pageNum	data	Integer		页数
	pageSize	data	Integer		页码
	pages	data	Integer		总共页数
	list	data	Object[]		
	tenantId	list	String		租户 id
	tenantName	list	String		租户 Name
	tenantCode	list	String		租户编码
	realCode	list	String		领域类型
	organizationType	list	String		租户类型(机构类型 1- 医疗机构 2-区域平台)
	gradeName	list	object		医院等级
	adminUser	list	String		管理员账号
	adminName	list	String		姓名
	devisionType	list	String		行政区划级别
	provinceCode	list	String		省
	cityCode	list	String		市
	districtCode	list	String		区
	tenantAddress	list	String		地址
	longitude	list	String		经度
	latitude	list	String		维度
	addDate	list	Date		创建时间（yyyy-MM-dd HH:mm:ss）
	modifyDate	list	Date		修改时间（yyyy-MM-dd HH:mm:ss）
	disabled	list	Integer		是否禁用 1-是 0-否
	delFlag	list	Integer		逻辑删除标志 1-是 0-否

示例：

1、请求地址：<https://xxctest.city.pingan.com/openapi/console/tenant/infos>

2、请求报文：

```
{
  "tenantId": "test",
  "startTime": "2020-01-19 00:00:00",
  "pageSize": 10,
  "pageNum": 1
}
```

3、返回报文：

```
{
  "retCode": 999999,
  "success": true,
  "retMsg": "成功",
  "data": {
    "pageNum": 5,
    "pageSize": 67,
    "pages": 9,
    "total": 256,
    "list": [
      {
        "tenantId": "fgffadfafd",
        "tenantName": "第一人民医院",
        "tenantCode": "hhhh",
        "realCode": "B",
        "organizationType": 1,
        "gradeName": "三级甲等",
        "adminUser": "test",
        "adminName": "降jjl",
        "devisionType": "1",
        "provinceCode": "659001000000",
        "cityCode": "659001000000",
        "districtCode": "659001000000",
        "tenantAddress": "xxxx 地址",
        "longitude": "121.9090",
        "latitude": "90.9999",
        "disabled": 1,
        "delFlag": 1
      }
    ]
  }
}
```

3.2.8 获取用户信息

备注：若传入用户名会根据用户名精确查询

接口代码	/console/user/info				
说明	获取用户信息				
请求方式	POST				
Content-Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求参数	tenantId		String	是	租户 id
	pageSize		Integer	是	页内数量（默认 10）
	pageNum		Integer	是	页码（默认 1）
	startTime		Date	是	最后修改时间（yyyy-MM-dd HH:mm:ss）
	accountName		String	否	用户名
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appld	head	String	是	appld—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	Object[]		响应对象
	total	data	Integer		总数
	pageNum	data	Integer		页数
	pageSize	data	Integer		页码
	pages	data	Integer		总共页数
	list	data	Object[]		
	userId	list	String		用户 id
	accountName	list	String		账号
	realName	list	String		真实姓名
	gender	list	Integer		性别（见字典）
	nickName	list	String		昵称
	headerIcon	list	String		头像
	tenantId	list	object		租户 id
	tenantName	list	String		租户名称
	isAdmin	list	String		是否为 admin 用户 1-是 0-否
	disabled	list	Integer		是否禁用 1-是 0-否
	delFlag	list	Integer		逻辑删除标志 1-是 0-否
	addDate	list	Date		创建时间（yyyy-MM-dd

					HH:mm:ss)
	modifyDate	list	Date		修改时间 (yyyy-MM-dd HH:mm:ss)

示例：

1、请求地址：<https://xxtest.city.pingan.com/openapi/console/user/info>

2、请求报文：

```
{
  "tenantId": "test",
  "startTime": "2020-01-19 00:00:00",
  "pageSize": 10,
  "pageNum": 1
}
```

3、返回报文：

```
{
  "retCode": 999999,
  "success": true,
  "retMsg": "成功",
  "data": {
    "pageNum": 5,
    "pageSize": 67,
    "pages": 9,
    "total": 256,
    "list": [
      {
        "userId": "fadafaf",
        "accountName": "eeee",
        "realName": "ceshi",
        "gender": 1,
        "nickName": "ddd",
        "headerIcon": "",
        "tenantId": "ddd",
        "tenantName": "test",
        "isAdmin": true,
        "disabled": 0,
        "delFlag": 0
      }
    ]
  }
}
```

3.2.9 获取员工信息

接口代码	/bs/employee/infos
------	--------------------

说明	获取员工信息				
请求方式	POST				
Content-Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求参数	tenantId		String	是	租户 id
	startTime		Date	是	最近修改时间 (yyyy-MM-dd HH:mm:ss)
	pageSize		Integer	是	页内数量 (默认 10)
	pageNum		Integer	是	页码 (默认 1)
	departmentCode		String	否	部门编码
	withSubDept		Integer	否	是否包含子部门 (1- 是 0- 否) (若 departmentCode 填了这个字段也必填)
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appId	head	String	是	appId—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	Object[]		响应对象
	total	data	Integer		总数
	pageNum	data	Integer		页数
	pageSize	data	Integer		页码
	pages	data	Integer		总共页数
	list	data	Object[]		
	tenantId	list	String		租户 ID
	employeeId	list	String		员工 ID
	employeeName	list	String		姓名
	employeeCode	list	String		工号
	gender	list	Integer		性别 (见字典)
	postName	list	String		职位名称
	postCode	list	Integer		职位编码 (见字典)
	titleCode	list	Integer		职称编码 (见字典)
	titleName	list	String		职称名称
	headerIcon	list	String		头像
	userId	list	String		用户 id
	accountName	list	String		用户账号
	departmentId	list	String		所属部门 id
	departmentName	list	String		所属部门名称
	delFlag	list	Integer		逻辑删除标志 1- 是 0- 否

	addDate	list	Date		创建时间 (yyyy-MM-dd HH:mm:ss)
	modifyDate	list	Date		修改时间 (yyyy-MM-dd HH:mm:ss)

示例：

1、请求地址：<https://xxctest.city.pingan.com/openapi/bs/employee/infos>

2、请求报文：

```
{
  "tenantId": "test",
  "startTime": "2020-01-19 00:00:00",
  "pageSize": 10,
  "pageNum": 1
}
```

3、返回报文：

```
{
  "retCode": 999999,
  "success": true,
  "retMsg": "成功",
  "data": {
    "pageNum": 5,
    "pageSize": 67,
    "pages": 9,
    "total": 256,
    "list": [
      {
        "tenantId": "dfdsdfd",
        "employeeId": "dfdfdfd",
        "employeeName": "ceshi",
        "employeeCode": "sddff",
        "gender": 1,
        "postCode": "230",
        "postName": "医生",
        "titleCode": "231",
        "titleName": "主任医师",
        "headerIcon": "",
        "userId": "dfdfdfd",
        "accountName": "test",
        "delFlag": 0,
        "departmentId": "fgsfsg",
        "departmentName": "fgsfsg"
      }
    ]
  }
}
```

3.2.10 获取部门信息

接口代码	/bs/department/infos				
说明	获取部门信息				
请求方式	POST				
Content-Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求参数	tenantId		String	是	租户 id
	startTime		Date	是	最新修改时间（yyyy-MM-dd HH:mm:ss）
	pageSize		Integer	是	页内数量（默认 10）
	pageNum		Integer	是	页码（默认 1）
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appld	head	String	是	appld—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	Object[]		响应对象
	total	data	Integer		总数
	pageNum	data	Integer		页数
	pageSize	data	Integer		页码
	pages	data	Integer		总共页数
	list	data	Object[]		
	tenantId	list	String		租户 ID
	departmentId	list	String		部门 ID
	parentId	list	String		上级 ID
	departmentName	list	String		名称
	departmentCode	list	String		编码
	departmentType	list	String		部门类型(1：院区 2:科室 ：3 部门)
	sectionType	list	String		科室属性(0:门诊, 1:住院)
	provinceCode	list	String		省
	cityCode	list	String		市
	districtCode	list	String		区
	tenantAddress	list	String		地址
	telephone	list	String		电话
	longitude	list	String		经度

	latitude	list	String		维度
	indexNum	list	Integer		排序
	levelNum	list	Integer		层级
	delFlag	list	Integer		逻辑删除标志 1-是 0-否
	addDate	list	Date		创建时间 (yyyy-MM-dd HH:mm:ss)
	modifyDate	list	Date		修改时间 (yyyy-MM-dd HH:mm:ss)

示例：

1、请求地址：<https://xxctest.city.pingan.com/openapi/bs/department/infos>

2、请求报文：

```
{
  "tenantId": "test",
  "startTime": "2020-01-19 00:00:00",
  "pageSize": 10,
  "pageNum": 1
}
```

3、返回报文：

```
{
  "retCode": 999999,
  "success": true,
  "retMsg": "成功",
  "data": {
    "pageNum": 5,
    "pageSize": 67,
    "pages": 9,
    "total": 256,
    "list": [
      {
        "tenantId": "hafhidajfahfiosa",
        "departmentId": "ahdiahfajdojif",
        "parentId": "djaojdopadfdf",
        "departmentName": "测试部门",
        "departmentCode": "",
        "departmentType": 1,
        "sectionType": 3,
        "provinceCode": "659001000000",
        "cityCode": "659001000000",
        "districtCode": "659001000000",
        "tenantAddress": "xxxx 地址",
        "telephone": "15899000011",
        "longitude": ""
      }
    ]
  }
}
```

```
        "latitude":"","  
        "indexNum":1,  
        "levelNum":1,  
        "delFlag":0  
    }  
}
```

3.2.11切换登录租户

接口代码	/sso/tenant/change				
说明	切换登录租户				
请求方式	POST				
Content - Type	application/json				
	节点名称	父节点名称	数据类型	必填	说明
加密前业务请求参数	tenantId		String	是	切换租户 id
	refreshToken		String	是	
Head 参数	节点名称	父节点名称	数据类型	必填	说明
	requestId	head	String	是	请求序列号---链路追踪
	timeStamp	head	String	是	时间戳—加密因子
	appId	head	String	是	appId—加密因子
	sign	head	String	是	签名
	signType	head	String	是	签名类型—加密因子
	ticket	head	String	是	票据—加密因子
	节点名称	父节点名称	数据类型	必填	说明
响应参数	data	data	object		响应对象
	accessToken	data	String		登录 token
	refreshToken	data	String		刷新 Token
	accessExpireDate	data	Integer		accessToken 有效时间
	refreshExpireDate	data	Integer		refreshtoken 有效时间
	userInfo	data	object		用户信息
	userId	userInfo	String		用户 id
	accountName	userInfo	String		账号
	realName	userInfo	String		真实姓名
	gender	userInfo	String		性别
	nickName	userInfo	String		昵称
	tenantId	userInfo	String		租户 id
	tenantName	userInfo	String		租户 Name

示例：

1、请求地址：<https://xxctest.city.pingan.com/openapi/bs/department/infos>

2、请求报文：

```
{
  "tenantId": "ghaaafafdfasnfns999900099",
  "refreshToken": "fasnfns999900099"
}
```

3、返回报文：

```
{
  "retCode": 999999,
  "success": true,
  "retMsg": "成功",
  "data": {
    "accessToken": "aaafafdfasnfns999fff",
    "refreshToken": "djfasnfns999fff",
    "accessExpireDate": 6000,
    "refreshExpireDate": 84600,
    "userInfo": {
      "userId": "aaafafdfasnfns999",
      "accountName": "test",
      "realName": "张三",
      "gender": 1,
      "nickName": "张思",
      "tenantId": "aaafafdfasnfns999ff",
      "tenantName": "珠海人民医院"
    }
  }
}
```

3.3 消息中心(Kafka)主题和消息体

3.3.1 用户消息推送

1、topic: uc_intergeration_user_info

2、消息体：JSON 格式

节点名称	父节点名称	数据类型	必填	说明
userId		String		用户 id
accountName		String		账号
realName		String		真实姓名
gender		Integer		性别（见字典）
nickName		String		昵称

headerIcon		String		头像
tenantId		object		租户 id
tenantName		String		租户名称
isAdmin		String		是否为 admin 用户 1-是 0-否
disabled		Integer		是否禁用 1-是 0-否
delFlag		Integer		逻辑删除标志 1-是 0-否
addDate		Date		创建时间 (yyyy-MM-dd HH:mm:ss)
modifyDate		Date		修改时间 (yyyy-MM-dd HH:mm:ss)

3.3.2 租户消息推送

1、topic: uc_intergeration_tenant_info

2、消息体：JSON 格式

节点名称	父节点名称	数据类型	必填	说明
tenantId	data	String		租户 id
tenantName	data	String		租户 Name
tenantCode	data	String		租户编码
realCode	data	String		领域类型
organizationType	data	String		租户类型(机构类型 1-医疗机构 2-区域平台)
gradeName	data	object		医院等级
adminUser	data	String		管理员账号
adminName	data	String		姓名
devisionType	data	String		行政区划级别
provinceCode	data	String		省
cityCode	data	String		市
districtCode	data	String		区
tenantAddress	data	String		地址
longitude	data	String		经度
latitude	data	String		维度
addDate	data	Date		创建时间 (yyyy-MM-dd HH:mm:ss)
modifyDate	data	Date		修改时间 (yyyy-MM-dd HH:mm:ss)
disabled	data	Integer		是否禁用 1-是 0-否
delFlag	data	Integer		逻辑删除标志 1-是 0-否

3.3.3 员工消息推送

1、topic：uc_intergeration_employee_info

2、消息体：JSON 格式

节点名称	父节点名称	数据类型	必填	说明
tenantId	list	String		租户 ID
employeeId	list	String		员工 ID
employeeName	list	String		姓名
employeeCode	list	String		工号
gender	list	Integer		性别（见字典）
postName	list	String		职位名称
postCode	list	Integer		职位编码（见字典）
titleCode	list	Integer		职称编码（见字典）
titleName	list	String		职称名称
headerIcon	list	String		头像
userId	list	String		用户 id
accountName	list	String		用户账号
departmentId	list	String		所属部门 id
departmentName	list	String		所属部门名称
delFlag	list	Integer		逻辑删除标志 1-是 0-否
addDate	list	Date		创建时间（yyyy-MM-dd HH:mm:ss）
modifyDate	list	Date		修改时间（yyyy-MM-dd HH:mm:ss）

3.3.4 部门消息推送

1、topic：uc_intergeration_department_info

2、消息体：JSON 格式

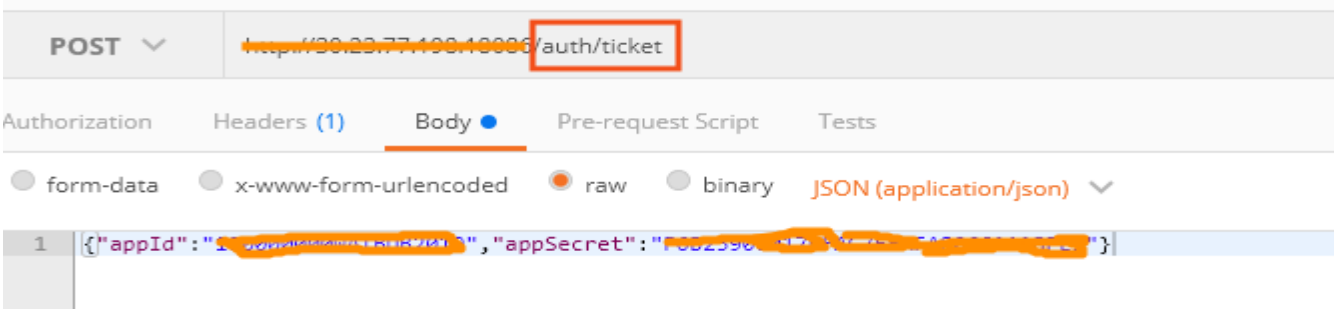
节点名称	父节点名称	数据类型	必填	说明
tenantId	list	String		租户 ID
departmentId	list	String		部门 ID
parentId	list	String		上级 ID
departmentName	list	String		名称
departmentCode	list	String		编码
departmentType	list	String		部门类型(1：院区 2:科室 ：3 部门)
sectionType	list	String		科室属性(0:门诊, 1:住院)
provinceCode	list	String		省
cityCode	list	String		市

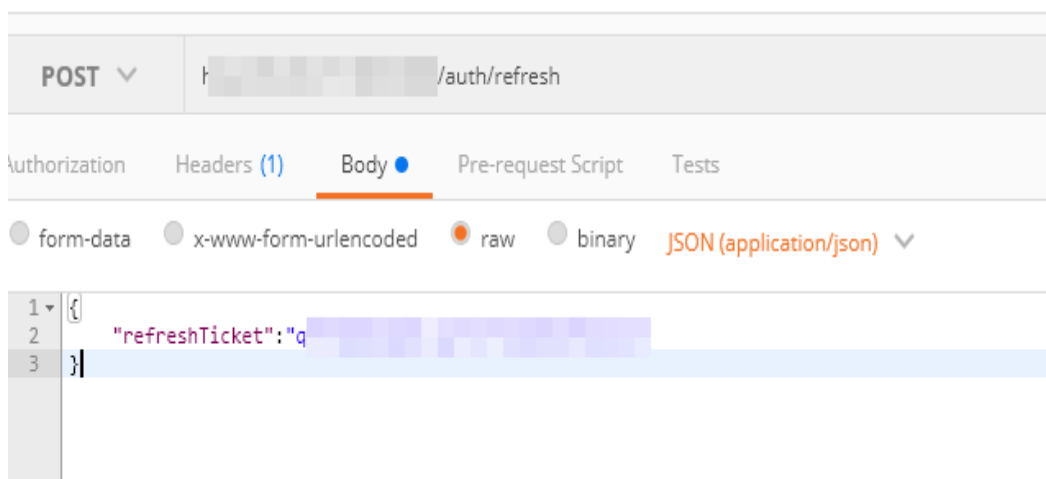
districtCode	list	String		区
tenantAddress	list	String		地址
telephone	list	String		电话
longitude	list	String		经度
latitude	list	String		维度
indexNum	list	Integer		排序
levelNum	list	Integer		层级
delFlag	list	Integer		逻辑删除标志 1-是 0-否
addDate	list	Date		创建时间 (yyyy-MM-dd HH:mm:ss)
modifyDate	list	Date		修改时间 (yyyy-MM-dd HH:mm:ss)

4 全流程调用过程图解

4.1 获取 ticket 示例

4.1.1 地址及 body

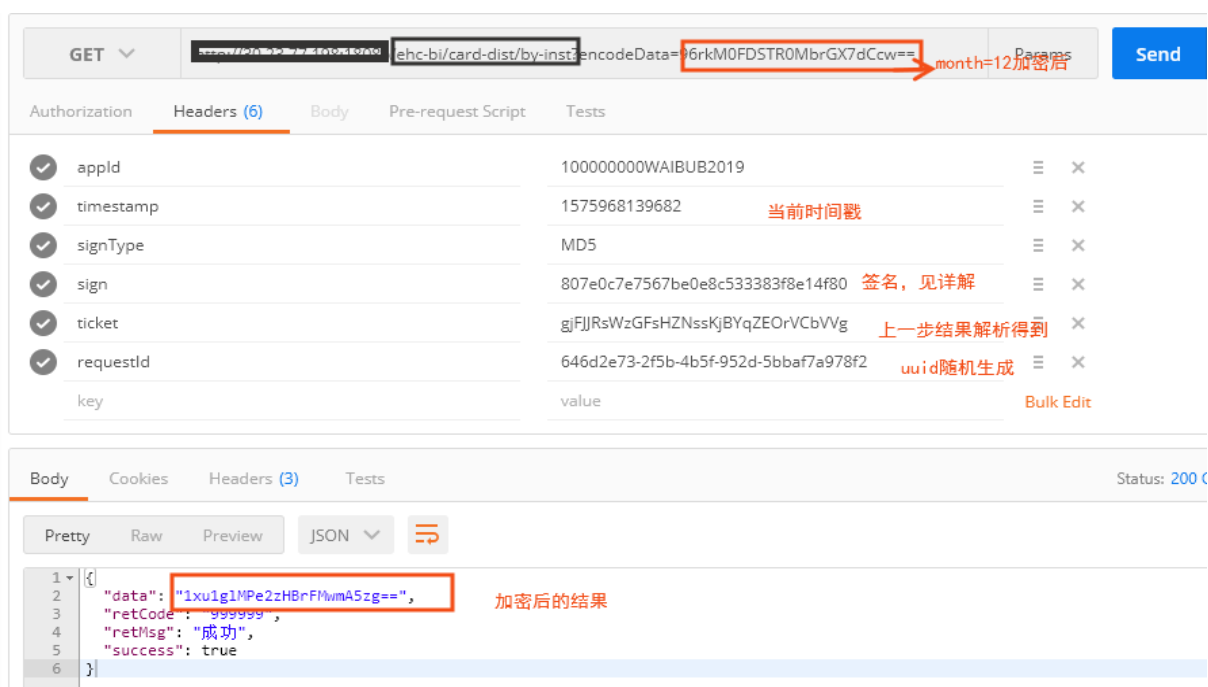




刷新 ticket 之后，原来的 ticket 的过期时间会更新延长，目前不会替换原 ticket。

4.3 业务接口调用示例

以查询 发卡数量医院 TOP10 为例：



签名 `sign = SignUtil.sign(signType, appld, ticket, ticketSecret, timeStamp, appSecret, appKey);`
 对上面的 data 进行解析 `SignUtil.decryptData(appSecret, "1xu1glMPe2zHBrFMwmA5zg==")`
 得到结果。

5 字典说明

5.1 性别

值	值含义
0	未知性别
1	男
2	女性
9	未说明性别

5.2 职位

值	值含义
230	医师
240	药剂
250	护理
260	技师

5.3 职称

值	值含义
231	主任医师
232	副主任医师

233	主治医师
234	医师
235	医士
241	主任药师
242	副主任药师
243	主管药师
244	药师
245	药士
251	主任护师
252	副主任护师
253	主管护师
254	护师
255	护士
261	主任技师
262	副主任技师
263	主管技师
264	技师
265	技士

5.4 行政区划数据

详见附件：省市区编码.xlsx

6 业务响应码说明

retCode	retMsg
UC0001	缺少必要参数
UC0008	未知租户
UC0044	子系统不存在
UC0048	refreshToken 已失效
UC0096	渠道配置为空
UC0104	获取票据失败

7 其他

暂无